

UDC: 004.896

Иерархическая когнитивная архитектура управления для мобильного робота

Аннотация

Рассматривается задача навигации мобильного робота с дифференциальным приводом в плоской среде с препятствиями и переменными свойствами поверхности. Предложена реализация трёхуровневой иерархической когнитивной архитектуры, объединяющей нижний уровень управления моторами на основе обучения с подкреплением, средний уровень коррекции на основе линейно-квадратичного регулятора и верхний уровень обхода препятствий с использованием известной карты местности. Проведён анализ функционального вклада каждого уровня архитектуры на двух семействах сценариев, охватывающих разные конфигурации препятствий и разные типы опорной поверхности. Показано, что распределение функций между уровнями архитектуры существенно зависит от структуры среды. Верхний уровень разгружает нижний при известной геометрии препятствий, но не компенсирует физических ограничений нижнего уровня при слабом сцеплении колёс с поверхностью. Все полученные результаты воспроизводимы с полной детерминированностью.

Ключевые слова: обучение с подкреплением, TD3, LQR, иерархическое управление, мобильная робототехника, STRL, residual policy, когнитивный агент

1. Введение

Автономная навигация мобильного робота в среде с препятствиями и переменными свойствами поверхности относится к задачам, в которых сходятся несколько направлений современных исследований: обучение с подкреплением, классическое управление, иерархическое планирование. Каждое из направлений действует на своём уровне абстракции и в своих допущениях, и вопрос об их интеграции в единую архитектуру остаётся открытым.

Один из устоявшихся подходов к этой задаче. Это трёхуровневая иерархическая когнитивная архитектура [1, 2, 3], в которой поведение агента распределено по трём уровням, действующим в существенно разных временных горизонтах. Нижний уровень управляет двигательной системой агента в реальном масштабе времени, средний обеспечивает локальную координацию движения на шкале

единиц секунд, верхний строит план поведения на длинных горизонтах на основе модели мира. В диссертационной работе [2] такая архитектура реализована для робототехнической платформы МП-РМ «Зарница» с Q -обучением на нижнем уровне. В работе [4] это направление развивается за счёт обработки мультимодальных сигналов когнитивным агентом.

В настоящей работе предложена реализация описанной трёхуровневой архитектуры для случая *непрерывного* управления мобильным роботом с дифференциальным приводом. Нижний уровень управления реализован нейросетевой стратегией, обученной методом TD3 [5]. Средний уровень — LQR-регулятор по линеаризованной модели бокового движения. Верхний уровень. Это модуль обхода препятствий с использованием известной карты Архитектура протестирована в физическом симуляторе PyBullet на платформе Husky.

Вклад работы. (1) Реализация трёхуровневой иерархической архитектуры, для непрерывного управления агентом в среде с препятствиями и переменными свойствами поверхности. (2) Эмпирический анализ распределения функций между уровнями: 65 ablation-ячеек ($7 \times 5 + 6 \times 5$), $n = 10$ запусков в каждой, 100 % воспроизводимость. Анализ количественно показывает, что верхний уровень с моделью среды разгружает нижний в задачах с известной геометрией препятствий, но не компенсирует физических ограничений нижнего уровня. (3) Открытый репозиторий с воспроизводимой реализацией.

Дальнейший текст организован следующим образом. Раздел 2 содержит обзор используемых методов и парадигм. Раздел 3 формулирует задачу и описывает архитектуру. Раздел 4 приводит результаты ablation-анализа. Раздел 5 обсуждает распределение функций между уровнями.

2. Обзор работ

2.1. Обучение с подкреплением для непрерывного управления. Для управления мобильным роботом в непрерывном пространстве действий табличные алгоритмы динамического программирования и DQN [6] неприменимы, поскольку функцию ценности $Q(s, a)$ невозможно параметризовать для бесконечного множества действий. В таких задачах используют параметрические аппроксиматоры, обучаемые методами актора-критика: DDPG [7], TD3 [5], SAC [8], PPO [9]. Метод TD3 выбран как off-policy схема с подтверждённой устойчивостью обучения на робототехнических задачах и доступной эталонной реализацией в `stable-baselines3` [10].

2.2. Residual policy и LQR в управлении мобильными роботами. Идея residual policy [11, 12] возникла как ответ на ограничения обучения с подкреплением без предварительных знаний и классических контроллеров: первое требует на порядки больше взаимодействий со средой, вторые теряют точность

при контактных взаимодействиях, в условиях трения и неучтённой нелинейности. Остаточная коррекция комбинирует обучаемую стратегию управления с классическим контроллером. Линейно-квадратичный регулятор (LQR) применяется к нелинейной кинематике дифференциального привода через линеаризацию в координатах ошибки. В настоящей работе реализован lateral LQR с упрощением $e_y = 0$, оправданным постановкой навигации к точечной цели.

2.3. Иерархический RL и waypoint-планирование. Разбиение задачи управления на уровни различной временной абстракции берёт начало в иерархическом подходе на основе опций [13]. Современные иерархические схемы обучения с подкреплением представлены HIRO [14], в котором цели задаются в исходном пространстве с переразметкой буфера переходов, и Director [15], в котором цели задаются в латентном пространстве модели мира. Между двумя крайностями располагаются схемы с фиксированной верхнеуровневой структурой: waypoint-планировщики и классические геометрические алгоритмы. Верхний уровень в настоящей работе реализован простейшим геометрическим способом, что позволяет изолировать вклад модельного знания, отделяя его от вклада сложности алгоритма планирования.

2.4. Иерархические архитектуры мобильных роботов. Многоуровневая организация управления мобильным агентом восходит к работам по поведенческой робототехнике 1990-х годов, прежде всего к систематизации трёхуровневой организации в [1]. В современной формулировке такая организация известна как *Strategic-Tactical-Reactive Levels* (STRL) и описана в работах [2, 3] как архитектура с тремя уровнями принятия решений, различающимися по горизонту планирования и используемой модели среды.

Параллельно развивается направление гибридизации обучения с подкреплением и методов оптимального управления. Один из ранних подходов. Это комбинирование обученной стратегии управления с классическим контроллером по схеме остаточной коррекции [11, 12], который обсуждался выше. К близким работам относятся подходы guided policy search, переносящие управление от model-based контроллера к нейросетевой стратегии [16], и работы по адаптивному управлению с гарантиями устойчивости [17], в которых классический регулятор служит safety-фильтром поверх обученной стратегии. Объединяющий эти направления принцип. Использовать обучение там, где модель среды неполна, и опираться на аналитические контроллеры там, где модель надёжна.

В отечественных исследованиях сходное направление развивает диссертация [2], реализующая трёхуровневую архитектуру для робототехнической платформы МП-РМ с Q -обучением [3] атомарных действий на нижнем уровне и иерархическим планированием на верхних. Работа [4] расширяет это направление за счёт обработки мультимодальных сигналов и переноса задач восприятия на современ-

ные нейросетевые модели. Настоящая работа развивает указанное направление в части непрерывного управления и количественного анализа вклада каждого уровня архитектуры в общую производительность.

3. Метод

3.1. Постановка задачи. Рассматривается задача навигации мобильного робота к случайной целевой точке на плоской арене с препятствиями известной геометрии и варьируемой поверхностью. Среда моделируется в PyBullet [18]; платформа Husky, четырёхколёсный робот с дифференциальным приводом. Управление подаётся на уровне нормированных скоростей колёсных пар.

Задача формализуется как марковский процесс принятия решений $\langle S, A, T, R, \gamma \rangle$ с непрерывными пространствами состояний и действий. Состояние агента в момент времени t образует вектор

$$s_t = (\mathbf{r}_t, \theta_t, \mathbf{v}_t, \omega_t, \mathbf{g}_t, \ell_t)^\top \in \mathbb{R}^{n_s}, \quad (1)$$

где $\mathbf{r}_t = (x_t, y_t)$ — координаты центра агента в рабочей области, θ_t — угол ориентации, $\mathbf{v}_t$ и ω_t — линейная и угловая скорости, $\mathbf{g}_t = \mathbf{r}_g - \mathbf{r}_t$ — относительное положение целевой точки, $\ell_t \in \mathbb{R}^{16}$ — вектор лидарных измерений (присутствует при наличии препятствий в среде). Действие $a_t \in [-1, 1]^2$ — нормированные целевые скорости левой и правой пар колёс, преобразуемые в физические угловые скорости $\omega_w \in [-10, 10]$ рад/с. Функция переходов T задаётся физикой PyBullet с дискретизацией $\Delta t_{\text{phys}} = 1/240$ с при шаге управления $\Delta t_{\text{ctrl}} = 1/20$ с. Дисконтирующий множитель $\gamma = 0.99$. Рабочая область — плоскость размером 10×10 м, целевая точка $\mathbf{r}_g$ сэмплируется на расстоянии от 2 до 5 м, радиус достижения цели — 0.4 м. Эпизод завершается по одному из условий: достижение цели, переворот агента ($|\theta_{\text{roll}}| > 60^\circ$), выход за пределы рабочей области, столкновение с препятствием или достижение лимита в 500 шагов управления (около 25 с). Функция вознаграждения — плотная и включает штраф за расстояние до цели, постоянный шаговый штраф, терминальный бонус за успех и терминальные штрафы за переворот и столкновение.

3.2. Иерархическая архитектура управления. В качестве организующего принципа используется парадигма многоуровневой иерархической архитектуры, восходящая к ранним работам по поведенческой робототехнике [1] и развитая для когнитивных агентов в работах [2, 3]. В настоящей работе эта архитектура адаптирована для непрерывного управления (рис. 1); три уровня выделяются следующим образом.

Нижний уровень. Это обученная методом TD3 стратегия управления $\pi_{\text{TD3}}(s) \rightarrow a$, действующая на каждом шаге управления. Опирается только на текущее наблюдение и в одиночку справляется с задачей навигации в простых средах.

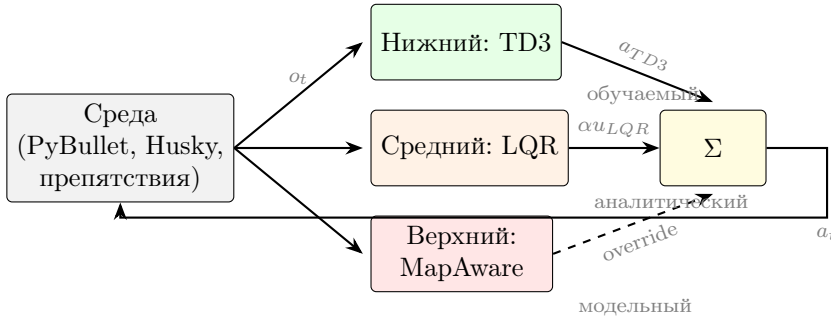


Рис. 1. Трёхуровневая иерархическая архитектура управления. Нижний уровень (TD3) выдаёт нормированное действие a_{TD3} на каждом шаге. Средний уровень (LQR-residual) выдаёт скалярную коррекцию u_{LQR} к угловой скорости, комбинируемую с весом α .

Верхний уровень (MapAware) при выполнении условия близости препятствия полностью переопределяет действие.

Средний уровень. Это LQR-регулятор с фиксированной матрицей усиления, построенный по линеаризованной модели бокового движения агента. Регулятор выдаёт скалярную коррекцию $u_{LQR} \in [-u_{\max}, u_{\max}]$ к угловой скорости, комбинируемую с действием нижнего уровня по схеме остаточной коррекции [11, 12]. Регулятор опирается на аналитическую модель кинематики, что обеспечивает гладкость рулевого управления независимо от качества обучения нижнего уровня.

Верхний уровень — модуль обхода препятствий с использованием известной карты их положений (MapAware). Уровень характеризуется индикаторной функцией активации $\mathbb{I}_{\text{map}}(s) \in \{0, 1\}$: при приближении агента к препятствию в передней полусфере на расстояние ниже порога $d_{\text{takeover}} = 1.5$ м индикатор принимает значение 1, при выходе за порог $d_{\text{release}} = 2.5$ м возвращается в 0 (двухпороговая логика с гистерезисом предотвращает дребезг управления на границе). Пороги d_{takeover} и d_{release} подобраны эмпирически из условия времени остановки агента на максимальной линейной скорости 2.8 м/с с запасом по тормозному пути. Содержательно этот уровень играет роль верхнего в трёхуровневой иерархической архитектуре с тем уточнением, что в текущей реализации он не предлагает последовательность подделей на длинном горизонте, а замещает управление в опасных конфигурациях. Полноценный планировщик является естественным направлением развития (раздел 6).

Конечное действие a_t на каждом шаге управления формируется правилом

$$a_t = \mathbb{K}_{\text{map}}(s_t) \cdot a_{\text{MapAware}}(s_t) + (1 - \mathbb{K}_{\text{map}}(s_t)) \cdot \text{clip} \left(a_{\text{TD3}}(s_t) + \alpha \cdot \begin{bmatrix} -1 \\ +1 \end{bmatrix} u_{\text{LQR}}(s_t), -1, 1 \right), \quad (2)$$

где $\mathbb{K}_{\text{map}}$ — индикатор активности верхнего уровня, знаки ± 1 преобразуют скалярную поправку u_{LQR} в дифференциальный сигнал на левую и правую пары колёс, $\alpha \in [0, 1]$ — фиксированный весовой коэффициент. В настоящей работе используется $\alpha = 0.2$. Это значение обеспечивает баланс между подавлением высокочастотных колебаний рулевого управления и сохранением стратегических решений нижнего уровня.

3.3. Нижний уровень: TD3. В качестве базовой стратегии управления используется TD3 [5], off-policy актор-критик с двойной оценкой функции ценности и задержанным обновлением актора. Стратегия π_ϕ и две функции ценности $Q_{\theta_1}, Q_{\theta_2}$ реализованы полносвязными нейронными сетями со скрытыми слоями размерности 400 и 300 и активацией ReLU. Выход стратегии пропускается через $\tanh$, что обеспечивает покомпонентное ограничение действия отрезком $[-1, 1]$. Обучение функций ценности минимизирует среднеквадратичную ошибку

$$\mathcal{L}(\theta_i) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[(Q_{\theta_i}(s, a) - y)^2 \right], \quad i = 1, 2, \quad (3)$$

с целевым значением

$$y = r + \gamma \min_{i=1,2} Q_{\theta'_i}(s', \pi_{\phi'}(s') + \varepsilon), \quad \varepsilon \sim \text{clip}(\mathcal{N}(0, \sigma_t), -c, c), \quad (4)$$

где θ'_i и ϕ' — параметры медленно обновляемых целевых сетей, σ_t и c — параметры target-smoothing. Стратегия обновляется с задержкой d итераций по градиенту

$$\nabla_\phi J(\phi) = \mathbb{E}_{s \sim \mathcal{D}} \left[\nabla_a Q_{\theta_1}(s, a) \Big|_{a=\pi_\phi(s)} \cdot \nabla_\phi \pi_\phi(s) \right]. \quad (5)$$

В работе использованы значения: размер минибатча 256, оптимизатор Adam со скоростью обучения 10^{-3} , коэффициент медленного обновления $\tau = 0.005$, задержка $d = 2$, $\sigma_t = 0.2$, $c = 0.5$, шум исследования $\mathcal{N}(0, 0.3)$, размер буфера переходов $\mathcal{D}$ составляет $2 \cdot 10^5$ переходов. Использована реализация [10].

Вектор наблюдения нижнего уровня в среде с препятствиями имеет вид

$$o_t = (\cos \theta_t, \sin \theta_t, \mathbf{v}_t, \omega_t, \mathbf{g}_t, \ell_t)^\top, \quad (6)$$

где первые два компонента кодируют ориентацию агента, $\mathbf{v}_t$ и ω_t — линейная и угловая скорости, $\mathbf{g}_t$ — относительное положение целевой точки, $\ell_t \in \mathbb{R}^{16}$ —

агрегированные лидарные измерения. Лидар моделируется по паспортным характеристикам сенсора LDS-01 платформы TurtleBot 3 Burger: 360 лучей с угловым шагом 1° вокруг агента, максимальная дальность сканирования 3.5 м (значение паспортное [19]). Для эффективности обучения стратегии 360 лучей агрегируются в 16 угловых секторов по 22.5° операцией поиска минимума: $\ell_t^{(k)} = \min_{j \in \mathcal{B}_k} d_t^{(j)}$, где $\mathcal{B}_k$ — индексы лучей k -го сектора, $d_t^{(j)}$ — дистанция, измеренная j -м лучом. Такая агрегация обеспечивает гарантированное обнаружение тонких препятствий, в отличие от низкоразрешённого 16-лучевого сканирования, при котором препятствие радиуса 0.3 м может проваливаться между соседними лучами. Наблюдение является привилегированным, поскольку эквивалентно идеальной локализации. Это допущение отделяет вопросы управления от вопросов восприятия.

3.4. Средний уровень: LQR-residual. В системе координат, связанной с прямой от робота к цели, состояние ошибки $e = [e_y, e_\theta]^\top$ (поперечное отклонение и отклонение курса) при фиксированной опорной скорости v_{ref} и малых отклонениях подчиняется линеаризованной модели:

$$\dot{e} = \begin{bmatrix} 0 & v_{\text{ref}} \\ 0 & 0 \end{bmatrix} e + \begin{bmatrix} 0 \\ -1 \end{bmatrix} \omega. \quad (7)$$

LQR находит закон управления $\omega = -Ke$, минимизирующий квадратичный функционал

$$J = \int_0^\infty (e^\top Q e + R \omega^2) dt, \quad (8)$$

где $Q = \text{diag}(2.0, 1.0)$, $R = 5.0$. Преобладание штрафа на управление ($R > Q_{ii}$) обеспечивает мягкие, не доминирующие коррекции. Матрица усиления K вычисляется однократно через численное решение алгебраического уравнения Риккати.

В реализации e_y обнуляется на каждом шаге. Опорная прямая переопределяется как линия от текущей позы агента к целевой точке, и накопленного поперечного отклонения относительно этой динамической прямой не возникает. Закон вырождается в $\omega = -k_\theta e_\theta$, структурно Р-регулятор по углу с коэффициентом из LQR-задачи. Это сознательное упрощение: в схеме residual policy LQR подавляет высокочастотные колебания курса, не вмешиваясь в стратегические решения нижнего уровня. Выход ω ограничивается $[-u_{\text{max}}, u_{\text{max}}]$, $u_{\text{max}} = 0.3$.

3.5. Верхний уровень: обход с использованием карты. Верхний уровень получает в качестве входных данных карту препятствий в виде списка $\{(o_i^x, o_i^y, r_i)\}$, где (o_i^x, o_i^y) — координаты центра i -го препятствия, r_i — его радиус. На каждом шаге управления вычисляются расстояния до краёв препятствий, находящихся в передней полусфере $\pm\pi/2$ относительно курса агента. Активация

уровня определяется двупороговой логикой с гистерезисом: индикатор $\mathcal{K}_{\text{map}}$ переключается с 0 на 1 при пересечении нижнего порога d_{takeover} снизу вверх и с 1 на 0 при пересечении верхнего порога d_{release} сверху вниз. Зазор между порогами ($d_{\text{release}} - d_{\text{takeover}} = 1.0$ м) предотвращает переключения управления вблизи границы зоны вмешательства, которые могли бы возникнуть при единственном пороге из-за измерительного шума и дискретизации управления. При активном уровне ($\mathcal{K}_{\text{map}} = 1$) управление задаётся прямолинейным движением со скоростью $v_{\text{sup}} = 0.4$ м/с и угловой скоростью

$$\omega_{\text{sup}} = k_{\text{turn}} \cdot \text{sign}(-\theta_{\text{nearest}}), \quad k_{\text{turn}} = 1.5, \quad (9)$$

где θ_{nearest} — угол на ближайшее препятствие в локальной системе координат агента, ограниченной передней полусферой.

Использованный геометрический алгоритм преднамеренно прост. В задачах с известной геометрией среды эффективное поведение часто достижимо без оптимизационного планировщика, а упрощение реализации позволяет изолировать вклад модельного знания о среде как такового. В работе также рассмотрен альтернативный сенсорный модуль обхода препятствий по лидарным дальностям (LidarAvoid), что позволяет сопоставить вклады модельного и сенсорного знания о среде.

4. Эксперименты

4.1. Конфигурация и метрики. Эксперименты проведены на сервере 8 ядер Ryzen 9 9950X / 16 ГБ RAM без GPU; PyBullet 3.2 в безголовом режиме, шесть параллельных сред через SubprocVecEnv. Сиды детерминированы: $\{200, 201, \dots, 209\}$; $n = 10$ запусков в каждой ячейке таблицы.

Основная метрика — доля успешных эпизодов (success rate). Дополнительно фиксируются длина эпизода, гладкость управления, максимальная линейная скорость, доля времени в верхнеуровневом режиме. Сравниваются пять архитектурных режимов: TD3, TD3+LQR, TD3+LidarAvoid, TD3+MapAvoid, TD3+MapAvoid+LQR.

4.2. Эксперимент А: множественные конфигурации препятствий. В первой серии оценивается зависимость от геометрии препятствий. Семь детерминированных конфигураций представляют качественно разные ситуации: одиночное препятствие на прямой, два препятствия со смещением, коридор из трёх, барьер с проходом, слалом, диагональное расположение цели, широкое препятствие.

Сценарий *diagonal_goal* без препятствий на прямой к цели решается всеми режимами и служит контролем — включение среднего или верхнего уровней не

Сценарий	TD3	+LQR	+Lidar	+Map	+Map+LQR
single_on_line	0/10	0/10	10/10	10/10	10/10
two_offset	0/10	0/10	10/10	10/10	10/10
three_corridor	0/10	0/10	0/10	10/10	10/10
barrier_with_gap	10/10	10/10	10/10	0/10	0/10
slalom	0/10	0/10	0/10	0/10	0/10
diagonal_goal	10/10	10/10	10/10	10/10	10/10
wide_obstacle	0/10	0/10	0/10	10/10	10/10

Таблица 1. Эксперимент А: доля успешных эпизодов (из 10 запусков на сидах 200–209) для семи конфигураций препятствий и пяти архитектурных режимов. Все ячейки 100 % детерминированы.

деградирует нижний. Сценарий *slalom* не решается ни одним режимом, фиксируя верхнюю границу применимости архитектуры в её текущем виде.

Распределение успеха в остальных пяти сценариях демонстрирует выраженное функциональное различие между сенсорным верхним уровнем (LidarAvoid) и модельным (MapAvoid). В сценариях *single_on_line* и *two_offset* оба варианта верхнего уровня обеспечивают полный успех, тогда как нижний и средний уровни в одиночку не справляются. Обученная стратегия идёт по прямой к целевой точке, не учитывая препятствие, а коррекция LQR подавляет высокочастотные колебания курса, но общую стратегию движения не меняет. В сценарии *three_corridor* полный успех достигает только модельный верхний уровень, что соответствует теоретической роли стратегического уровня в задачах с известной структурой среды.

Обратный паттерн наблюдается в сценарии *barrier_with_gap*, где прямой путь к цели проходит через узкий проход в барьере. Здесь полностью нижеуровневые варианты и нижний с сенсорным верхним уровнем обеспечивают 100 % успех, тогда как все варианты с модельным верхним уровнем не справляются. Причина — в логике MapAvoid: широкий радиус упреждающего вмешательства заставляет верхний уровень обходить барьер целиком, тогда как геометрический проход в нём проходим напрямую. В *wide_obstacle* обратная асимметрия — широкое препятствие аппроксимировано в карте цилиндром малого радиуса, и MapAvoid не активируется на нужной дистанции. Сенсорный верхний уровень также не справляется, поскольку поведенческая логика выбора объезда неустойчива при широких препятствиях. Эти два сценария указывают, что полезность каждого варианта верхнего уровня зависит от соответствия его внутренней модели реальной геометрии среды.

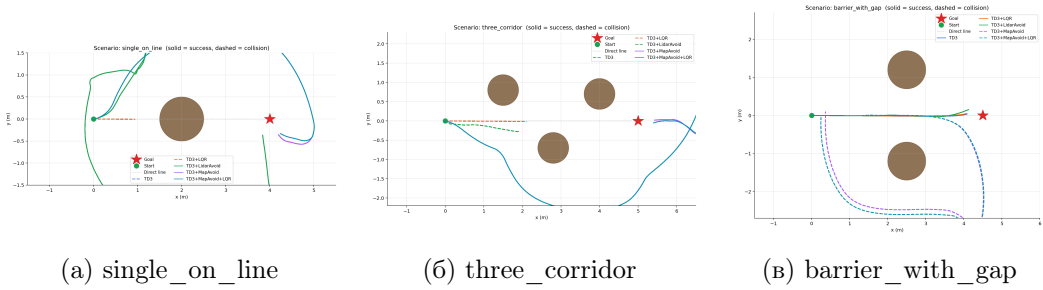


Рис. 2. Траектории робота на трёх характерных сценариях эксперимента А (вид сверху). Старт обозначен зелёным кружком, цель — красным, серые круги — препятствия известной геометрии. Линии разных цветов показывают траектории пяти архитектурных режимов. (а) оба варианта верхнего уровня успешны, нижний уровень в одиночку сталкивается с препятствием. (б) только модельный верхний уровень справляется. (в) прямой проход через цель закрывается модельным верхним уровнем, но успешно проходит лидарным.

Воспроизводимость. Результаты в таблице 1 полностью детерминированы. Каждая ячейка имеет однозначный исход для всех сидов из набора. Детерминированность является свойством сцены и обусловлена детерминированностью PyBullet при фиксированных стартовых условиях, фиксированными весами стратегии управления и отсутствием стохастичности в верхних уровнях архитектуры.

4.3. Эксперимент В: переменные свойства поверхности. Во второй серии оценивается влияние свойств опорной поверхности. Реализованы три условия: *NORMAL* (трение 0.9), *ICE* (трение 0.05) и *SAND* (трение 0.6 с дополнительным сопротивлением качению). Каждое условие тестируется в двух конфигурациях: *empty* и *with_obstacle*, $n = 10$.

Поверхность	Конфиг.	TD3	+LQR	+Lidar	+Map	+Map+LQR
NORMAL	empty	10/10	10/10	10/10	10/10	10/10
NORMAL	+ obst.	0/10	0/10	0/10	10/10	10/10
ICE	empty	10/10	10/10	10/10	10/10	10/10
ICE	+ obst.	0/10	0/10	0/10	0/10	10/10
SAND	empty	10/10	10/10	10/10	10/10	10/10
SAND	+ obst.	0/10	0/10	10/10	10/10	10/10

Таблица 2. Эксперимент В: доля успешных эпизодов (из 10) для трёх типов поверхности и двух конфигураций. Модель TD3 v3 обучена на детерминированном сценарии с препятствием на *NORMAL* и используется во всех ячейках без переобучения.

В пустой арене нижний уровень одинаково успешен на всех трёх поверхностях, хотя максимальная развитая скорость различается (NORMAL: ~ 0.6 м/с, ICE: ~ 0.3 м/с, SAND: ~ 0.5 м/с). Стратегия, обученная на NORMAL, переносит навык базовой навигации на ICE и SAND, автоматически снижая скорость движения, что соответствует физическим характеристикам поверхностей.

В присутствии препятствия на NORMAL и SAND модельный верхний уровень решает задачу полностью (10/10), что согласуется с экспериментом А. На SAND с препятствием успех демонстрируют также сенсорный верхний уровень и его комбинации со средним уровнем, тогда как чистый нижний режим (TD3 / TD3+LQR) не справляется. Принципиальный результат — сценарий ICE с препятствием, где ни один режим без среднеуровневой коррекции не решает задачу. При попытке обхода низкое трение приводит к проскальзыванию и столкновению. Полный успех (10/10) достигается только в полной трёхуровневой конфигурации TD3+MapAvoid+LQR, в которой средний уровень (LQR-регулятор) обеспечивает плавность курсового управления, необходимую для удержания траектории на скользкой поверхности. Это эмпирически подтверждает функциональное взаимодополнение трёх уровней архитектуры: ни один уровень в отдельности не достаточен для задач с физическими ограничениями нижнего уровня.

5. Обсуждение

Нижний уровень компетентен в условиях свободного движения. На всех трёх типах поверхности в пустой рабочей области стратегия управления достигает полного успеха, автоматически адаптируя линейную скорость к условиям сцепления без переобучения. Это соответствует теоретической роли нижнего уровня в иерархической архитектуре. Это управление на коротком горизонте без обращения к модели среды.

Средний уровень не меняет стратегических решений. Включение LQR-регулятора поверх нижнего уровня не меняет долю успеха ни в одной ячейке таблиц 1, 2. LQR подавляет высокочастотные колебания курса, что фиксируется в дополнительных измерениях гладкости управления, однако не влияет на способность преодолевать препятствия. Это согласуется с теоретической ролью среднего уровня в иерархической архитектуре [2, 3].

Верхний уровень разгружает нижний при адекватной модели среды. Эксперимент А показывает, что в сценариях *single_on_line* и *two_offset* включение верхнего уровня обеспечивает полный успех там, где нижний уровень не справляется. Это типичная демонстрация полезности верхнего уровня в иерархической архитектуре. Обратный эффект в сценариях *barrier_with_gap* и *wide_obstacle* указывает на чувствительность верхнего уровня к параметрам геометрической

модели среды. Полезность модельного знания не абсолютна и зависит от точности представления реальной геометрии препятствий.

Модельное знание не компенсирует физических ограничений нижнего уровня в условиях слабого сцепления. В эксперименте В на сценарии ICE+obstacle полный успех достигается только полной трёхуровневой архитектурой. Без среднего уровня LQR ни один режим, включая режим с верхним уровнем, не решает задачу: верхний уровень предлагает корректную траекторию обхода, однако низкое трение приводит к проскальзыванию агента независимо от выбранной стратегии. Только добавление среднего уровня, обеспечивающего гладкость рулевого управления, позволяет удерживать траекторию.

Общая трактовка. Полезность верхних уровней архитектуры функционально определена структурой среды и качеством работы нижних уровней и не является абсолютной. В свободном движении полезность верхних уровней нулевая. В задачах с известной геометрией среды верхний уровень становится критичным. В задачах с физическими ограничениями на нижнем уровне средний и верхний уровни взаимодополняют друг друга, и только их совместное применение обеспечивает решение.

Ограничения. Использован простейший геометрический алгоритм на верхнем уровне без оптимизационного планирования. Нижний уровень обучен с использованием привилегированного наблюдения. Реальная робототехническая платформа не задействована.

6. Заключение

В работе реализована и эмпирически исследована трёхуровневая иерархическая архитектура управления мобильным агентом с дифференциальным приводом, соотносимая с трёхуровневой иерархической архитектурой [2, 3]. Нижний уровень реализован обученной методом TD3 стратегией управления. Средний уровень реализован LQR-регулятором в схеме остаточной коррекции с упрощением $e_y = 0$. Верхний уровень реализован модулем обхода препятствий с использованием известной карты.

Ablation-анализ на семи конфигурациях препятствий и трёх типах поверхности количественно характеризует распределение функций. Средний уровень не меняет долю успеха, но улучшает гладкость. Верхний уровень разгружает нижний при известной геометрии препятствий, но не справляется при слабом сцеплении колёс. Все результаты воспроизведены со 100 % детерминированностью.

Направления развития. (1) Верхний уровень в полноценной стратегической формулировке. Имеется в виду оптимизационный планировщик последовательно-сти подцелей на длинном горизонте. Реализация и сравнение с использованным

здесь упрощённым вариантом требуют отдельной работы. (2) Интеграция с мультимодальной модельной компонентой в духе [4] для естественного дополнения архитектуры визуальной модальностью. (3) Реальная робототехническая платформа. Это ключевое направление развития группы [2]. Перенос из PyBullet на платформу с ROS требует адаптации наблюдения (восстановление позы из SLAM) и проверки устойчивости к sim-to-real разрыву.

ЛИТЕРАТУРА

1. Gat E. On three-layer architectures // Artificial Intelligence and Mobile Robots / D. Kortenkamp, R. Bonasso, R. Murphy (eds.). AAAI Press, 1998. P. 195–210.
2. Киселёв Г. А. Интеллектуальная система планирования поведения коалиции робототехнических агентов со STRL архитектурой: дис. ... канд. техн. наук. Москва, 2020. 117 с.
3. Киселёв Г. А., Панов А. И. Иерархическое психологически-инспирированное планирование для интеллектуальных динамических систем // Известия РАН. Теория и системы управления. 2020. № 1. С. 123–137.
4. Вейценфельд Д. А., Киселёв Г. А. Метод синтеза поведения когнитивного агента на основе обработки мультимодальных сигналов // Моделирование и анализ данных. 2024. Т. 14. № 4. С. 45–62.
5. Fujimoto S., van Hoof H., Meger D. Addressing function approximation error in actor-critic methods // Proc. of ICML. 2018. P. 1587–1596.
6. Mnih V., Kavukcuoglu K., Silver D. et al. Human-level control through deep reinforcement learning // Nature. 2015. V. 518. P. 529–533.
7. Lillicrap T., Hunt J., Pritzel A. et al. Continuous control with deep reinforcement learning // Proc. of ICLR. 2016.
8. Haarnoja T., Zhou A., Abbeel P., Levine S. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor // Proc. of ICML. 2018. P. 1861–1870.
9. Schulman J., Wolski F., Dhariwal P., Radford A., Klimov O. Proximal policy optimization algorithms // arXiv preprint arXiv:1707.06347. 2017.
10. Raffin A., Hill A., Gleave A. et al. Stable-Baselines3: Reliable reinforcement learning implementations // Journal of Machine Learning Research. 2021. V. 22. № 268. P. 1–8.
11. Silver T., Allen K., Tenenbaum J., Kaelbling L. Residual policy learning // arXiv preprint arXiv:1812.06298. 2018.
12. Johannink T., Bahl S., Nair A. et al. Residual reinforcement learning for robot control // Proc. of ICRA. 2019. P. 6023–6029.

13. Sutton R. S., Precup D., Singh S. Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning // Artificial Intelligence. 1999. V. 112. P. 181–211.
14. Nachum O., Gu S., Lee H., Levine S. Data-efficient hierarchical reinforcement learning // Proc. of NeurIPS. 2018. P. 3303–3313.
15. Hafner D., Lee K.-H., Fischer I., Abbeel P. Deep hierarchical planning from pixels // Proc. of NeurIPS. 2022.
16. Levine S., Finn C., Darrell T., Abbeel P. End-to-end training of deep visuomotor policies // Journal of Machine Learning Research. 2016. V. 17. P. 1–40.
17. Cheng R., Orosz G., Murray R. M., Burdick J. W. End-to-end safe reinforcement learning through barrier functions for safety-critical continuous control tasks // Proc. of AAAI. 2019. P. 3387–3395.
18. Coumans E., Bai Y. PyBullet, a Python module for physics simulation for games, robotics and machine learning. 2016–2021. URL: <http://pybullet.org>
19. Robotis Co., Ltd. LDS-01 (360 Laser Distance Sensor): Spec sheet // URL: https://emanual.robotis.com/docs/en/platform/turtlebot3/appendix_lds_01/. 2017.